

CAF — Product

Aggregated export — 2026-06-16. Source markdown in repo is unchanged.

Bundle id: 01-caf-product

Included: docs/CAF_PRODUCT_PITCH.md

CAF — Product pitch

Audience: Leadership, investors, potential partners, and operators evaluating CAF as a platform — not engineers implementing it.

One-liner: CAF is a **content operating system** that turns research and signals into reviewed, publishable social content — with quality gates, human oversight, and a learning loop built in.

The problem

Teams scaling social content (carousels, reels, scripted video) hit the same wall:

- **Ad-hoc AI** produces inconsistent output with no lineage, no caps, and no audit trail.
- **Spreadsheets and scripts** don't scale across brands, platforms, or reviewers.
- **Expensive renders** (video, image generation) run before anyone checks copy or risk.
- **Learning stays tribal** — what worked last month doesn't systematically improve next month's prompts or planning.

CAF treats content production as **operational software**: structured intake → planned work → generation → QC → render → human review → publish → learn.

What CAF is

CAF (Content Automation Framework) is a **multi-tenant content pipeline platform**.

Layer	What it is
CAF Core	Backend brain — Postgres holds truth; APIs orchestrate the full funnel
Review app	Operator workbench — approve, reject, rework, publish
Media services	Carousel renderer + video assembly — production-grade PNG/video output
Admin	Project config, inputs processing, flow engine, learning rules

It is **not** a single ChatGPT prompt or a no-code wrapper. It is **infrastructure for repeatable, governed content operations**.

Who CAF is for

Persona	How they use CAF
Content ops / social teams	Run weekly production cycles from signal packs; review queue in one place
Brand / growth leads	Set strategy, caps, risk policies, and brand constraints per project
Editors & reviewers	Approve or send back work with structured rework — without losing history
Engineering / platform	Extend flows, integrate publishers, wire learning from performance data
Agencies managing multiple brands	One platform, many projects (tenants), isolated config and queues

What makes CAF different

Typical approach	CAF approach
One-off LLM calls	Content jobs with task_id, drafts, and full revision history
Hope the copy is safe	QC runtime — checklists + risk policies before expensive render
Manual file handoffs	generation_payload — one contract from plan → review → publish
"We'll remember what worked"	Learning rules — planning boosts and prompt guidance from evidence
Copy-paste top performers	Top-performer mimic — visual patterns from archived winners, fresh copy
Black-box automation	Human gate by default after QC; editorial decisions are first-class data

Core capabilities (product view)

1. Structured intake

Upload research (Excel, evidence pipelines, scrapers) → **signal packs** with typed ideas and lineage. No more unstructured "folder of insights."

2. Intelligent planning

Decision engine scores candidates, applies caps and suppression, selects prompts and routes — and records **why** (decision traces).

3. Controlled generation

Per-brand prompts, output schemas, tenant config, and learning-injected guidance. Generation is **configurable**, not improvised.

4. Quality before cost

Automated QC and keyword risk scanning run **before** carousel renders and video jobs — protecting budget and brand safety.

5. Media production

Carousels (Handlebars + Puppeteer), HeyGen/Sora video paths, ffmpeg stitch/mux — assets land in storage with stable IDs.

6. Human review & rework

Review app for operators: approve, reject, needs edit. Rework orchestration regenerates what changed without corrupting lineage.

7. Publishing intent

Publication placements record what to post, when, and outcomes — with optional Meta Graph execution or external workers (n8n, manual).

8. Learning loop

Performance metrics, editorial analysis, and LLM post-approval reviews feed **learning rules** that improve future planning and prompts.

9. Top-performer mimic (optional)

Ingest high-performing creative, analyze visuals, recreate **look-and-feel** with new copy — carousel and single-image flows.

End-to-end story (60 seconds)

```
Research in → Signal pack → Plan jobs → Generate copy → QC  
→ Render media → Human review → Publish → Measure → Learn  
→ (next run)
```

Every step is **data in Postgres** and **HTTP APIs** — auditable, replayable, and integrable.

Business outcomes

- **Throughput** — Run many jobs per cycle with caps, not chaos.
 - **Consistency** — Same brand voice, schemas, and QC every time.
 - **Cost control** — Block bad copy before video/image spend.
 - **Compliance posture** — Risk policies, banned words, honest QC status reporting.
 - **Institutional memory** — Learning rules and run context snapshots document what was used to generate content.
 - **Speed to iterate** — Rework paths and prompt labs without rebuilding pipelines from scratch.
-

Deployment model

- **Self-hosted or cloud** — Core on Fly.io (or similar); Review on Vercel; media workers as separate services.
 - **Bring your own keys** — OpenAI, HeyGen, BFL, Supabase, Meta — CAF orchestrates; you own credentials.
 - **Multi-brand** — Projects (`SNS` , `Cuisina` , ...) share platform code, isolate config and queues.
-

Honest maturity note

CAF is a **real production platform**, not a demo. Some flow types are fully wired (carousel, several video paths, mimic when enabled). Others are registered for future work (certain image product flows). The pitch is the **architecture and operating model** — individual flow maturity is documented in engineering guides.

Next steps for evaluators

Goal	Read next
Full product walkthrough	CAF COMPLETE PRODUCT GUIDE.md
Technical architecture	ARCHITECTURE.md or CAF CORE COMPLETE GUIDE.md
Stakeholder summary	PROJECT OVERVIEW.md
Bootstrap a stack	REBUILD FROM DOCS.md

CAF — Content Automation Framework. Pipeline truth in Postgres; humans stay in the loop; learning compounds over time.

Included: docs/CAF_COMPLETE_PRODUCT_GUIDE.md

CAF — Complete product guide

Purpose: The definitive **product-level** guide to what CAF is, what it does, who uses it, and how every major capability fits together. For engineering detail, see [CAF CORE COMPLETE GUIDE.md](#). For a short pitch, see [CAF PRODUCT PITCH.md](#).

Convention: “CAF” means the platform; **CAF Core** is the backend in this repository.

Table of contents

1. [Executive summary](#)
 2. [Platform components](#)
 3. [The content funnel](#)
 4. [Core concepts](#)
 5. [Upstream: inputs & research](#)
 6. [Planning & decision engine](#)
 7. [Generation & drafts](#)
 8. [Quality checks & risk](#)
 9. [Rendering & media](#)
 10. [Human review & rework](#)
 11. [Publishing](#)
 12. [Learning & improvement](#)
 13. [Top-performer mimic & creative intelligence](#)
 14. [Flow types \(what content CAF can produce\)](#)
 15. [Multi-tenant projects](#)
 16. [Operator surfaces](#)
 17. [Typical workflows](#)
 18. [Integrations & external systems](#)
 19. [What CAF is not](#)
 20. [Documentation map](#)
-

1. Executive summary

CAF (Content Automation Framework) is a **content pipeline platform** for teams that produce scaled social content — carousels, reels, scripted video, and related formats.

It implements a full operating loop:

Signal Pack → **Candidates** → **Decision Engine** → **Content Jobs** → **LLM Drafts** → **QC** → **Rendering** → **Review** → **Publishing** → **Learning**

Unlike a script that calls an LLM once, CAF:

- Stores **every stage** in PostgreSQL (`caf_core` schema).
 - Keys work off stable IDs (`task_id` , `run_id`) for traceability.
 - Enforces **quality and risk gates** before expensive media steps.
 - Keeps **humans in the loop** with structured editorial decisions and rework.
 - Feeds **performance and editorial evidence** back into future planning and prompts.
-

2. Platform components

Component	Location	Role
CAF Core API	Repo root (<code>src/</code>)	Orchestration, business logic, Postgres, HTTP APIs
Review app	<code>apps/review/</code>	Next.js operator UI — client of Core, not DB of record
Carousel renderer	<code>services/renderer/</code>	Puppeteer + Handlebars → slide PNGs
Video assembly	<code>services/video-assembly/</code>	ffmpeg → stitch, mux, subtitles
Media gateway	<code>services/media-gateway/</code>	Single port exposing renderer + assembly
Admin UI	Core <code>/admin</code>	Inputs, processing, project config, flow engine

Source of truth: Postgres `caf_core` , especially `content_jobs` and `generation_payload` .

3. The content funnel

```
flowchart LR
    IN["Inputs / research"]
    SP["Signal pack"]
    PL["Planning"]
    GEN["Generation"]
    QC["QC"]
    REN["Render"]
    REV["Review"]
    PUB["Publish"]
    LR["Learning"]
```

IN --> SP --> PL --> GEN --> QC --> REN --> REV --> PUB --> LR
LR --> PL

Stage	Product meaning
Inputs	Evidence, scrapers, spreadsheets → structured rows
Signal pack	Curated research bundle for one production cycle
Planning	Which ideas become jobs, which flows, which prompts
Generation	LLM produces structured copy / scripts per flow schema
QC	Automated checks + risk keywords before render
Render	Carousel PNGs, HeyGen video, scene clips, mimic images
Review	Human approve / reject / needs edit
Publish	Placement records + optional platform execution
Learning	Rules and evidence improve the next cycle

4. Core concepts

Concept	Plain language
Project	A brand/tenant (e.g. SNS). Strategy, flows, bans, learning rules.
Signal pack	Research bundle attached to a run — ideas, summaries, visual guidelines.
Run	One production cycle for a project (e.g. SNS_2026W09).
Candidate	One idea × one flow type — scored in memory during planning.
Content job	Atomic unit of work. Key: <code>task_id</code> . Holds <code>generation_payload</code> .
Draft	One LLM attempt for a job (revision history).
Asset	Rendered file (image, video, audio) linked to a job.
Editorial review	Human decision: approved, rejected, or needs edit.
Publication placement	Intent to post on a platform + outcome status.
Learning rule	Structured insight that changes planning or prompts.

ID patterns (examples): `task_id` =

`SNS_2026W09__Instagram__FLOW_CAROUSEL__row0002__v1` . Full detail:
[DOMAIN MODEL.md](#).

5. Upstream: inputs & research

Before a signal pack exists, CAF can ingest **evidence**:

Input	How	Product outcome
Excel / XLSX	Upload API or CLI	Legacy signal pack rows
Evidence imports	Admin /admin/inputs	Normalized rows in Postgres
Scrapers (Apify)	Admin scrapers tab	Social posts → same evidence shape
Insights passes	Admin /admin/processing	Broad LLM, top-performer vision (image, carousel, video)
Build signal pack	Processing API	Curated jobs_json / ideas for planning
Creative Intelligence	Ingest APIs	Archived top-performer media + vision analysis

Review app role: Pipeline pages can **inspect** evidence and signal-pack ideas — processing controls stay in Core admin.

Detail: [CAF INPUTS PIPELINE ROADMAP.md](#), [CREATIVE INTELLIGENCE.md](#).

6. Planning & decision engine

When a **run starts**, CAF:

1. Loads materialized planner rows from the signal pack (`planned_jobs_json`).
2. Builds **candidates** (idea × enabled flow types).
3. **Scores** them (confidence, platform fit, novelty, past performance).
4. Applies **caps** (max carousel/video jobs per run, daily limits).
5. Applies **suppression** and **learning boosts** (rank/score adjustments).
6. Selects **prompts** and **routes** per flow.
7. Creates `content_jobs` rows — one per selected plan row.
8. Persists a **decision trace** and **run context snapshot** (what was used to plan).

Product value: Transparent, repeatable planning — not “generate everything and hope.”

Detail: [layers/decision-engine.md](#).

7. Generation & drafts

For each job in `PLANNED` / `GENERATING` status:

- Resolves **prompt templates** from the flow engine (per project + flow type).

- Builds a **creation pack** (signal context, brand, strategy — mimic jobs filter to a single idea).
- Injects **learning guidance** into prompts where rules apply.
- Calls **OpenAI** (or placeholder mode for testing).
- Validates output against **output schemas** (configurable strictness).
- Stores result in **job_drafts** and merges **generated_output** into **generation_payload** .

Special paths: scene bundles, product video, mimic pre-copy (reference resolution before LLM).

Detail: [layers/generation.md](#), [GENERATION GUIDANCE.md](#).

8. Quality checks & risk

QC runs after generation, before render (by default).

Check source	What it does
QC checklists	Required keys, lengths, regex, slide counts — per flow
Risk policies	Keyword scan on generated JSON (scoped per flow type)
Brand banned words	Project-level word bans merged into same scan

Outcomes: pass, block, rework route, or route to human review. Clean passes still typically require **human review** (`CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC`).

Important honesty: Project **risk rules** in admin are **not** the same as QC-enforced **risk policies** — see [RISK RULES.md](#).

Detail: [QUALITY CHECKS.md](#).

9. Rendering & media

After QC, jobs move to **rendering** based on flow type:

Format	Production path
Carousel	HTTP to carousel renderer → PNG slides → Supabase assets
HeyGen video	Video agent / avatar API → poll → MP4 asset
Scene assembly	Sora or other clip provider → video-assembly stitch/mux
Mimic image/carousel	Image provider (BFL, etc.) + template/DocAI text overlays

Core tracks **render state** (e.g. HeyGen `video_id`) to avoid double-submission on retries.

Detail: [layers/rendering.md](#), [VIDEO FLOWS.md](#), [MIMIC FLOWS COMPLETE GUIDE.md](#).

10. Human review & rework

Operators use the **Review app** (or `/v1` APIs):

Decision	Meaning
APPROVED	Ready for publish (may trigger post-approval LLM review)
REJECTED	Discarded; lineage preserved
NEEDS_EDIT	Rework — script, slides, HeyGen prompt, carousel slides, etc.

Rework orchestrator regenerates only what changed, preserving `task_id` and history.

Review supports carousel slide editing, mimic layer positions, visual guidelines, HeyGen prompt tweaks, and debug bundles for support.

Detail: [layers/review-rework.md](#).

11. Publishing

Publication placements record:

- Which job (`task_id`) posts where
- Schedule and status (`draft` → `scheduled` → `published` / `failed`)
- Platform outcome (post ID, errors)

Executors:

Mode	Behavior
<code>none</code>	External worker (manual, n8n) completes via API
<code>dry_run</code>	Immediate fake completion (CI / infra tests)
<code>meta</code>	Core calls Meta Graph (Facebook Page + Instagram)

Stable publish payload shape: `GET .../n8n-payload` (name is historical).

Detail: [layers/publishing.md](#).

12. Learning & improvement

CAF closes the loop in **two ways**:

A. Prompt guidance (generation)

Active learning rules with generation/guidance actions are compiled into text appended to LLM prompts — “don’t quote verbatim” institutional hints.

B. Planning boosts (selection)

Rules with rank/score actions affect **which jobs get planned** — not the prompt text.

Additional learning inputs:

- Performance metrics ingestion (CSV-style)
- Editorial analysis cron (optional)
- Post-approval LLM reviews → **upstream recommendations** → learning observations
- Run context snapshots — fingerprint of prompts + guidance used at plan time

Detail: [layers/learning.md](#), [GENERATION GUIDANCE.md](#).

13. Top-performer mimic & creative intelligence

Optional advanced capability for brands with archived winners:

1. **Ingest** top-performing posts (Creative Intelligence).
2. **Analyze** visuals (vision models) → styling cues on signal packs.
3. **Plan** mimic flow jobs (`FLOW_TOP_PERFORMER_MIMIC_IMAGE` / `FLOW_TOP_PERFORMER_MIMIC_CAROUSEL`).
4. **Resolve** reference frames before copy generation.
5. **Generate** fresh copy (OpenAI) matching structure.
6. **Render** via image providers + carousel overlays (HBS / DocAI).

Modes: single full-bleed image, template background + text, per-slide carousel visual.

Requires `MIMIC_IMAGE_ENABLED` and provider keys.

Detail: [MIMIC FLOWS COMPLETE GUIDE.md](#), [CREATIVE INTELLIGENCE.md](#). Future text-placement QA: [MIMIC TEXT PLACEMENT AUTOMATION.md](#).

14. Flow types (what content CAF can produce)

Flows are registered in the **flow engine** per project. Families include:

Family	Examples	Maturity
Carousel	<code>FLOW_CAROUSEL</code>	Production
Video / reel	Various <code>FLOW_*</code> video kinds	Production (provider-dependent)

Product video	FLOW_PRODUCT_*	Production
Scene assembly	Multi-clip + stitch	Production when configured
Mimic	FLOW_TOP_PERFORMER_MIMIC_*	Production when enabled
Image product	FLOW_IMG_*	Registered; not fully wired
Offline / legacy	Some variation names	Excluded from auto-pipeline

Allowed flows are configured per **project** in `allowed_flow_types` .

15. Multi-tenant projects

Each **project** (brand) has isolated:

- Strategy defaults and brand constraints (voice, banned words)
- Platform constraints (carousel typography, etc.)
- Allowed flow types and HeyGen config
- Learning rules (project-scoped)
- Risk rules (admin) and risk policies (QC-enforced)
- Brand assets, carousel templates, Meta integrations
- Mimic render settings (optional)

URLs use **project_slug** (e.g. `/v1/runs/SNS/...`).

16. Operator surfaces

Surface	User	Capabilities
Review app	Editors, ops	Queue, approve/reject/rework, publish, learning UI, run logs
Core /admin	Admins, engineers	Inputs, processing, project config, flow engine, diagnostics
HTTP /v1 APIs	Integrations	Full pipeline automation
CLIs	Engineers	Migrate, seed, process-run, xlsx ingest, scrapers

17. Typical workflows

Weekly content cycle (classic XLSX)

1. Upload signal pack XLSX → create run.
2. Materialize jobs → start run → process → render.
3. Review queue → approve winners.
4. Create publication placements → publish.

5. Ingest performance → update learning rules.

Evidence-driven cycle (inputs pipeline)

1. Upload evidence XLSX or run scraper.
2. Run insights (broad + top-performer tiers).
3. Build signal pack from selected rows.
4. Same plan → generate → QC → render → review → publish → learn.

Single-job rework

1. Reviewer marks **NEEDS_EDIT** with notes/overrides.
 2. Rework orchestrator re-runs generation and/or render.
 3. Job returns to review queue with preserved lineage.
-

18. Integrations & external systems

System	Role
OpenAI	Generation, QC helpers, insights, mimic copy, optional Sora clips
HeyGen	Avatar / video agent renders
BFL / DashScope / NVIDIA	Mimic image edit providers
Supabase Storage	Asset URLs (slides, video, scenes)
Meta Graph	Optional in-Core publishing
Apify	Scraper orchestration for inputs
Google Document AI	OCR for mimic carousel text placement
n8n / webhooks	External publish workers via placements API
Fly.io / Vercel	Deploy Core, media gateway, Review app

Env reference: `ENV_AND_SECRETS_INVENTORY.md` , `.env.example` .

19. What CAF is not

- **Not** a generic chatbot or content calendar alone.
 - **Not** a replacement for your creative strategy — it **operationalizes** it.
 - **Not** fully no-code — flow engine and prompts are configurable but engineering extends behavior.
 - **Not** guaranteeing every registered flow type is production-ready — see flow maturity in §14.
 - **Not** storing secrets in git — keys live in env / vault.
-

20. Documentation map

Audience	Document
Pitch / evaluators	CAF PRODUCT PITCH.md
This guide	CAF COMPLETE PRODUCT GUIDE.md
Stakeholder summary	PROJECT OVERVIEW.md
Engineering merged ref	CAF CORE COMPLETE GUIDE.md
Architecture	ARCHITECTURE.md
ChatGPT / external repos	EXTERNAL CONTEXT PACK.md
Rebuild stack	REBUILD FROM DOCS.md
IDs & tables	DOMAIN MODEL.md , DATABASE SCHEMA.md
API examples	API REFERENCE.md
AI coding assistants	AGENTS.md

CAF — from research to reviewed, publishable content, with gates, humans, and learning at every step.

Included: docs/PROJECT_OVERVIEW.md

CAF Core — Project overview

This document explains **what CAF Core is**, **what problems it solves**, and **how the pieces fit together** at a level suitable for stakeholders and new engineers.

What is CAF Core?

CAF (Content Automation Framework) is a **content pipeline platform**. **CAF Core** is the backend: a **Fastify API + PostgreSQL** application that owns **operational truth** for content production—research signals, planned work units (**content jobs**), AI-generated drafts, quality checks, rendered media, human review decisions, publication scheduling, and learning artifacts.

Companion apps and services talk to Core over HTTP:

Piece	Role
CAF Core (this repo root)	API, business logic, <code>caf_core</code> schema
Review app (<code>apps/review</code>)	Next.js operator UI; not the database of record

Carousel renderer (services/renderer)	Puppeteer + Handlebars → slide images
Video assembly (services/video-assembly)	ffmpeg stitch/mux/subtitles → video files
Media gateway (services/media-gateway)	Single port for renderer + assembly

Core is **self-contained**: planning, jobs, QC, rendering, review, publishing, and learning all live in this repo and Postgres. Companion services only add media rendering and the operator UI.

What problem does it solve?

Teams that produce **scaled social content** (carousels, reels, scripted video) need:

1. **Structured intake** — research / signals turned into candidates with typed fields and lineage.
2. **Controlled generation** — prompts, schemas, tenant config, caps, suppression.
3. **Quality gates** — automated checks + risk keyword policies before expensive render.
4. **Human review** — approvals, rework, overrides without corrupting lineage.
5. **Media production** — carousel PNGs, HeyGen/Sora/ffmpeg pipelines, assets in storage.
6. **Publishing intent** — placements, schedules, outcomes (Meta or external workers).
7. **Learning** — rules and evidence so future runs can score better and prompts can include guidance.
8. **Top-performer mimic (optional)** — recreate visual patterns from archived high performers with fresh copy (`FLOW_TOP_PERFORMER_MIMIC_*` flows; requires Creative Intelligence ingest + `MIMIC_IMAGE_ENABLED`).

CAF Core implements that **as data in Postgres** and **HTTP APIs**, not as a single monolithic LLM script.

Core concepts (plain language)

- **Project** — A tenant (brand). Holds strategy, brand constraints, allowed flow types, learning rules.
- **Signal pack** — A bundle of research (often from an `.xlsx` upload). Feeds **candidates**.
- **Run** — One production cycle for a project, linked to a signal pack. Groups all jobs from that intake.
- **Content job** — The **atomic unit of work**. Keyed by `task_id`. Carries `generation_payload` (the main JSON contract: prompts, LLM output, QC, render hints, publish URLs).
- **Draft** — One LLM attempt for a job (`job_drafts`).
- **Asset** — Rendered file (image/video) linked to a job (`assets` , often Supabase URLs).
- **Publication placement** — Scheduled or completed publish intent per platform (`publication_placements`).

Typical workflow

1. Upload or CLI-ingest a **signal pack**; create a **run**.
2. **Start** the run → orchestrator builds **planned jobs** in **content_jobs** (**PLANNED**).
3. **Process** the run (or single job) → LLM generation → **QC** → diagnostics → **render** (carousel/video) → **IN_REVIEW** .
4. Operators use the **Review app** (or APIs) → **approve**, **reject**, or **needs edit** (rework).
5. **Publications** API records what to post and outcomes; **learning** routes ingest metrics and rules.

For a **technical** walkthrough (files, tables, boundaries), see [ARCHITECTURE.md](#).

Who should read what

Audience	Doc
Pitch / investors / evaluators	docs/CAF_PRODUCT_PITCH.md
Complete product guide (what CAF does)	docs/CAF_COMPLETE_PRODUCT_GUIDE.md
Product / ops / leadership (short)	This file
External LLM / other repository	docs/EXTERNAL_CONTEXT_PACK.md + Tier 1 bundle
Rebuild / bootstrap	docs/REBUILD_FROM_DOCS.md
Engineers implementing features	docs/CAF_CORE_COMPLETE_GUIDE.md (single file), or docs/ARCHITECTURE.md, docs/layers/README.md, README.md, docs/API_REFERENCE.md
Database / schema	docs/DATABASE_SCHEMA.md
Domain IDs & lifecycles	docs/DOMAIN_MODEL.md
Lifecycle & states	docs/LIFECYCLE.md
QC / guidance / risk behavior	docs/QUALITY_CHECKS.md, docs/GENERATION_GUIDANCE.md, docs/RISK_RULES.md
Top-performer mimic	docs/MIMIC_FLOWS_COMPLETE_GUIDE.md, docs/MIMIC_IMAGE_FLOWS.md, docs/CREATIVE_INTELLIGENCE.md
AI assistants / tooling	AGENTS.md (repo root)
Environment & secrets	docs/USER_INPUT_AND_SECRETS.md, ENV_AND_SECRETS_INVENTORY.md

Relationship to “CAF” branding

Marketing may call CAF a “content operating system.” In **this repository**, treat that as a **description of the pipeline**, not a guarantee that every subsystem is equally mature: some **flow types** are fully wired (carousel, several video paths); others are **registered but not yet implemented** (e.g. certain image product flows—see `src/domain/product-flow-types.ts`).

See also

- [CAF PRODUCT PITCH.md](#) — product pitch for evaluators
- [CAF COMPLETE PRODUCT GUIDE.md](#) — **full product guide**
- [EXTERNAL CONTEXT PACK.md](#) — **ChatGPT / external repo bundle index**
- [REBUILD FROM DOCS.md](#) — bootstrap guide
- [DOMAIN MODEL.md](#) — entities and IDs
- [DATABASE SCHEMA.md](#) — table catalog
- [CAF CORE COMPLETE GUIDE.md](#) — **everything in one document** (merged from split docs)
- [ARCHITECTURE.md](#) — layers, modules, lifecycle
- [LIFECYCLE.md](#) — state machines
- [TECH STACK.md](#)
- [layers/README.md](#) — one doc per layer
- [QUALITY CHECKS.md](#), [GENERATION GUIDANCE.md](#), [RISK RULES.md](#)
- [API REFERENCE.md](#) — HTTP examples
- [VIDEO FLOWS.md](#) — video-specific behavior
- [MIMIC FLOWS COMPLETE GUIDE.md](#) — top-performer mimic (full)
- [MIMIC IMAGE FLOWS.md](#) — mimic quick reference
- [CREATIVE INTELLIGENCE.md](#) — top-performer ingest